2026학년도 2학기 언어데이터과학

제16강 HuggingFace 데이터 활용

박수민

서울대학교 인문대학 언어학과

2026년 11월 9일 월요일

지난 시간에 배운 것

1. 동형어 번호를 떼고 논문의 후보 목록을 다시 만들었다(2,652개).
2. 사전의 기준값을 세어 논문의 표를 나란히 읽었다(2음절 21.9%, ㅂ 58.2%).
3. 같은 수를 다시 얻으려면 버전과 방법을 남겨야 한다.

오늘의 목표

1. HuggingFace Hub에서 데이터셋을 찾고, 카드에서 무엇을 어떻게 모았는지 읽을 수 있다.
2. `load_dataset`으로 설정, 분할, 버전을 정해 불러와 표로 바꿀 수 있다.
3. 데이터를 만든 과정이 수에 남는다는 것을 세어서 확인할 수 있다.
4. 라이선스를 확인하고, 남이 같은 데이터를 얻도록 출처를 적을 수 있다.

사전 준비

터미널 입력 (Codespaces)

```
cd /workspaces/LDS2026
pip install datasets
```

노트북 16-huggingface.ipynb의 첫 셀

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from datasets import load_dataset

sns.set_theme(style='whitegrid', font='NanumGothic')
```

노트북은 저장소 맨 위에 만든다. 10강의 `ratings.txt`와 `kiwipiepy`를 다시 쓴다.

데이터를 구하는 길

강의	데이터	구한 방법	파일
10강	NSMC	`wget`으로 GitHub에서	TSV
11강	모두의 말뭉치	신청, 승인, zip 내려받기	JSON
14강	표준국어대사전	zip 내려받기	XML
오늘	**HuggingFace Hub**	**코드 한 줄**	**Parquet**
17강	내가 만드는 코퍼스	웹에서 직접 모으기	내가 정한다

기말프로젝트의 주제는 정했는데 데이터가 없다면

누군가 이미 공개해 둔 데이터가 있을지 모른다.

15강 끝에서 남긴 물음

남이 만든 데이터셋을 쓰려면 그 데이터셋에 대해 **무엇이 적혀 있어야** 할까?

1 **HuggingFace Hub**
데이터셋 찾기
데이터셋 카드

2 코드로 불러오기

3 KLUE-NLI 읽기

4 가져다 쓰기 전에

5 마무리

한국어 데이터셋 찾기



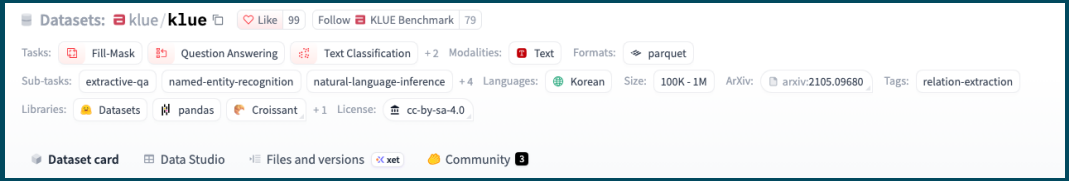
Hub는 모델과 데이터셋을 올리고 내려받는 곳이다. 왼쪽 [Languages]에서 [Korean]을 고르면 **3,379개**가 나온다.

https://huggingface.co/datasets (2026-11-03 캡처)

목록을 읽을 때

allenai/c4처럼 여러 언어가 섞인 것도 한국어 태그를 달고 있다. 태그는 올린 사람이 적고, 내려받은 수는 품질을 말해 주지 않는다.

데이터셋의 첫 화면



`https://huggingface.co/datasets/klue/klue` (2026-11-03 캡처)

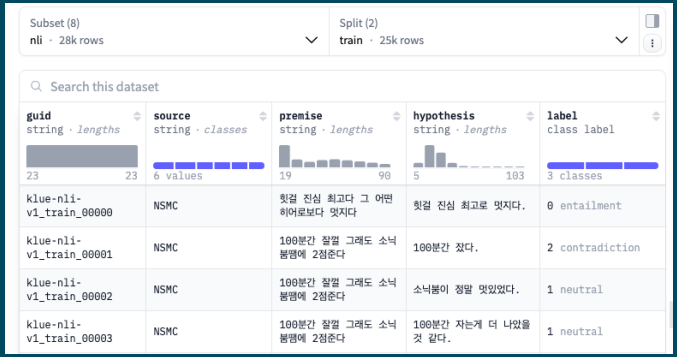
KLUE는 한국어 이해 능력을 평가하려고 만든 과제 8개의 묶음이다(Park et al. 2021).

`klue/klue` 저장소 이름. GitHub처럼 **올린 이/데이터셋**으로 적는다

태그 과제, 언어, 크기, 파일 형식, 라이선스(`cc-by-sa-4.0`)

탭 카드(Dataset card), 표 보기(Data Studio), 파일과 커밋(Files and versions)

내려받기 전에 표를 본다



Subset (8) nli · 28k rows

Split (2) train · 25k rows

Search this dataset

guid string · lengths 23 23	source string · classes 6 values	premise string · lengths 19 90	hypothesis string · lengths 5 103	label class label 3 classes
klue-nli-v1_train_00000	NSMC	힛걸 진심 최고다 그 어떤 히어로보다 멋지다	힛걸 진심 최고로 멋지다.	0 entailment
klue-nli-v1_train_00001	NSMC	100분간 잘껄 그래도 소닉 붐땜에 2점준다	100분간 잤다.	2 contradiction
klue-nli-v1_train_00002	NSMC	100분간 잘껄 그래도 소닉 붐땜에 2점준다	소닉붐이 정말 멋있었다.	1 neutral
klue-nli-v1_train_00003	NSMC	100분간 잘껄 그래도 소닉 붐땜에 2점준다	100분간 자는게 더 나았을 것 같다.	1 neutral

Subset은 묶음 안의 과제 하나로 8개다. Split은 나누어 둔 부분으로 2개다. 열마다 이름, 자료형, 분포가 보인다.

Data Studio 탭, `nli`, `train` (2026-11-03 캡처)

오늘은 `nli`를 쓴다. 전제(`premise`)와 가설(`hypothesis`)이 짝을 이룬다. 첫 줄의 출처 NSMC는 10강의 영화평이다.

데이터셋 카드

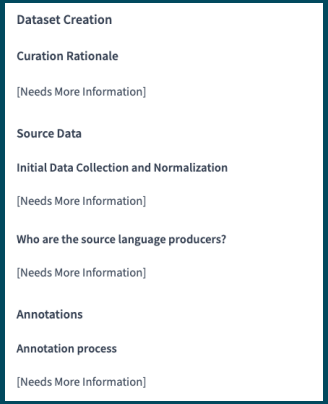
카드는 저장소의 README.md다

2강에서 만든 README.md와 같은 파일이다. 맨 위의 머리말이 앞 쪽의 태그가 되고, 본문이 카드가 된다.

카드의 항목	답하는 물음
Dataset Summary, Languages	무엇을 모았나, 어떤 언어인가
Data Fields, Data Splits	열은 무엇이고, 어떻게 나누었나
Dataset Creation	누가, 어디서, 어떤 절차로 만들었나
Considerations for Using	쓸 때 조심할 점은 무엇인가
Licensing, Citation	어떻게 써도 되고, 어떻게 밝히나

15강의 물음에 대한 Hub의 답이다. 다만 카드는 올린 사람이 채우는 글이다.

카드의 빈칸은 다른 곳에서 채운다



Dataset Creation

Curation Rationale

[Needs More Information]

Source Data

Initial Data Collection and Normalization

[Needs More Information]

Who are the source language producers?

[Needs More Information]

Annotations

Annotation process

[Needs More Information]

klue/klue 카드 (2026-11-03 캡처)

KLUE 카드에서 만든 과정은 비어 있다. 카드는 `[Needs More Information]`이 기본값인 틀이다.

그 대신 맨 위에 **논문과 GitHub 저장소의 링크**가 있다. KLUE의 만든 과정은 논문에 자세히 있다. 카드가 비었다고 곧 돌아서지 말고, 카드가 가리키는 곳을 읽는다. 뒤에서 그 논문을 함께 읽는다.

1 HuggingFace Hub

2 **코드로 불러오기**
load_dataset
안 되는 것 두 가지

3 KLUE-NLI 읽기

4 가져다 쓰기 전에

5 마무리

load_dataset

노트북 입력

```
nli = load_dataset('klue/klue', 'nli')
print(nli.keys())
print(nli['train'])
```

실행 결과

```
dict_keys(['train', 'validation'])
Dataset({
    features: ['guid', 'source', 'premise', 'hypothesis', 'label'],
    num_rows: 24998
})
```

`'nli'`는 설정(화면의 Subset)이다. 결과는 분할 이름을 열쇠로 갖는 사전 `DatasetDict`다.

한 행

노트북 입력 — 한 행은 dict

```
for key, value in nli['train'][0].items():
    print(key, value)
```

실행 결과

```
guid klue-nli-v1_train_00000
source NSMC
premise 힛걸 진심 최고다 그 어떤 히어로보다 멋지다
hypothesis 힛걸 진심 최고로 멋지다.
label 0
```

label이 수로 적혀 있다. 0이 무엇인지는 다음 쪽에서 꺼낸다.
앞 셀에서 unauthenticated requests, IProgress 경고가 나와도 된다. 공개 데이터는 로그인 없이 받는다.

라벨 번호와 이름

노트북 입력

```
names = nli['train'].features['label'].int2str
print(names(0), names(1), names(2))
```

실행 결과

```
entailment neutral contradiction
```

features 각 열의 자료형. label은 이름표가 달린 범주다(ClassLabel)

int2str 번호를 이름으로 바꾸는 함수. 괄호 없이 함수 자체를 names에 담았다

번호는 데이터셋마다 다르게 매겨진다. 0이 무엇인지 짐작하지 말고 꺼내어 확인한다.

분할

노트북 입력 — 분할 하나만 부르면 Dataset이 온다

```
dev = load_dataset('klue/klue', 'nli', split='validation')
print(type(dev).__name__, dev.num_rows)
```

실행 결과

```
Dataset 3000
```

분할	행	쓰임
훈련(train)	24,998	규칙을 배운다
검증(validation, 논문의 dev)	3,000	고르고 조정한다
시험(test)	3,000	마지막 평가. **Hub에 없다**

논문은 30,998쌍, Hub에는 27,998쌍이다. 쓰임은 18강에서 경험한다.

어디에 내려받았나

터미널 입력

```
du -sh ~/.cache/huggingface/*
```

실행 결과

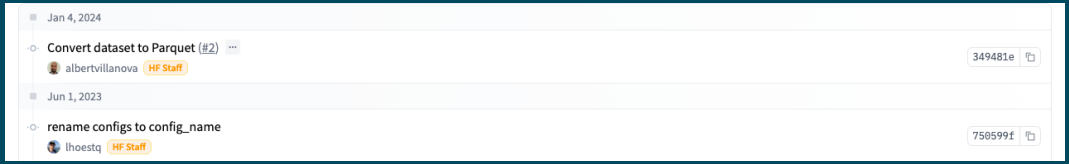
```
6.1M    /home/codespace/.cache/huggingface/datasets
2.0M    /home/codespace/.cache/huggingface/hub
48K     /home/codespace/.cache/huggingface/xet
```

hub/는 받은 원본(.parquet, 표를 담은 이진 파일), datasets/는 바로 읽는 표(.arrow)다.

저장소 바깥이다

다시 부르면 여기서 읽는다. git에 잡히지 않으니 내 저장소에는 **불러오는 코드**만 남는다.

버전 못 박기



Hub의 저장소는 git 저장소다(2강). `revision`이 없으면 최신 커밋을 받는다.

노트북 입력

```
nli = load_dataset('klue/klue', 'nli',
                   revision='349481ec73fff722f88e0453ca05c77a447d967c')
print(nli['train'].num_rows, nli['validation'].num_rows)
```

```
24998 3000
```

Hub에 있어도 불러와지지 않는 것

노트북 입력 — 일부러 부르는 줄. 노트북에는 넣지 않는다

```
load_dataset('e9t/nsmc')
```

실행 결과 — 오류 메시지의 마지막 줄

```
RuntimeError: Dataset scripts are no longer supported, but found nsmc.py
```



e9t/nsmc의 첫 화면 (2026-11-03 캡처)

Hub의 NSMC 저장소에는 데이터가 없고 내려받는 코드 `nsmc.py`가 있다. `datasets`는 이제 남의 코드를 돌리지 않는다. 이럴 때는 **원본**에서 받는다(10강의 `wget`).

같은 코드, 다른 수

노트북 입력 — 12강의 \b를 한 번 더

```
s = pd.Series(['완전 좋아', '이건 완전히 달라'])
print(s.dtype.storage, s.str.count(r'\b완전\b').tolist())
```

실행 결과

```
pyarrow [0, 0]
```

12강이라면 `[1, 0]`이다. **오류도 경고도 없이** 0이다. 12강의 '완전' 8,636번도 0이 된다.

pip install datasets가 함께 설치한 pyarrow

pandas 3은 pyarrow가 있으면 `str.count`, `str.contains`의 정규표현식을 그쪽 엔진으로 돌린다. 거기서 \b와 \w는 한글을 글자로 치지 않는다.

한 줄로 되돌린다

노트북 입력 — 표를 만들기 **전에** 둔다

```
pd.set_option('mode.string_storage', 'python')
s = pd.Series(['완전 좋아', '이건 완전히 달라'])
print(s.dtype.storage, s.str.count(r'\b완전\b').tolist())
```

실행 결과

```
python [1, 0]
```

이 Codespace의 다른 노트북도 마찬가지다

12·13강 노트북을 다시 돌리거나 기말프로젝트에서 \b, \w를 쓴다면 **첫 셀에 이 줄을 넣는다.** 지난 노트북의 저장된 출력은 그대로라서, 다시 돌리기 전에는 알아채지 못한다.

환경도 남긴다

노트북 입력

```
import datasets, pyarrow
print(pd.__version__, datasets.__version__, pyarrow.__version__)
```

실행 결과

```
3.0.5 5.1.0 26.0.0
```

버전을 적는 까닭

데이터도 코드도 그대로인데 설치한 패키지 때문에 수가 바뀌었다. 15강에서 남기자고 한 ‘버전과 방법’에는 **데이터의 버전**뿐 아니라 **패키지의 버전**도 들어간다. 여러분의 화면과 수가 다르면 이 줄부터 비교한다.

1 HuggingFace Hub

2 코드로 불러오기

3 **KLUE-NLI 읽기**
가설은 시켜서 쓴 문장이다
부정 표현 세기
검증 세트에서는

4 가져다 쓰기 전에

5 마무리

표로 바꾸고, 세 가지 관계를 읽는다

노트북 입력

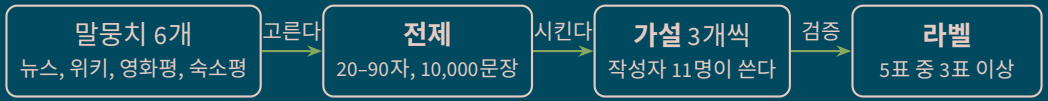
```
train = nli['train'].to_pandas()
train['name'] = train['label'].map(names)
p = '1월 여행의 가장 큰 걱정은 추위였어요.'
print(train.loc[train['premise'] == p, ['hypothesis', 'name']])
```

실행 결과

```
                    hypothesis           name
327  1월 여행에서 추위 따위는 걱정되지 않았어요.  contradiction
328      2월 여행에서도 추위가 가장 걱정돼요.        neutral
329   추위는 1월 여행에서 가장 큰 걱정이었어요.     entailment
```

전제가 참일 때 가설도 반드시 참이면 함의(entailment), 반드시 거짓이면 모순(contradiction), 어느 쪽도 될 수 있으면 중립(neutral)이다.

어떻게 만들었나



전제는 찾은 말, 가설은 시켜서 쓴 말

작성자는 전제 하나를 보고 함의, 중립, 모순 문장을 **하나씩** 쓴다. 가설은 이 과제를 위해 태어난 문장이다.

논문이 미리 걱정한 것

라벨마다 버릇이 남는다(*annotation artifact*, 주석의 흔적). 논문은 모순과 함께 나오는 부정 표현을 예로 들고, 같은 단어를 되풀이하지 말라고 지시했다.

출처: Park et al. (2021), arXiv:2105.09680, 20–21쪽

전제의 출처를 따라가 본다

노트북 입력 — 10강의 영화평과 대조한다

```
ratings = pd.read_csv('data/nsmc/ratings.txt', sep='\t')
kn = train.loc[train['source'] == 'NSMC', 'premise'].drop_duplicates()
print(len(kn), kn.isin(ratings['document']).sum())
found = ratings['document'].str.contains('헛걸 진심', na=False)
print(ratings.loc[found, 'document'].tolist())
```

실행 결과

```
1645 856
['미안하다 이제서야 봐서.. 헛걸 진심 최고다 그 어떤 히어로보다 멋지다']
```

10강의 영화평에 그대로 있는 전제는 절반쯤이고, 첫 전제도 리뷰에서 잘라 낸 **문장**이다.
영화평의 표본이 아니다.

모순을 쓰라고 하면

노트북 입력 — 가설에 부정 표현이 있는가

```
NEG = r'않|없|못| 안 '
train['neg'] = train['hypothesis'].str.contains(NEG)
print((train.groupby('name')['neg'].mean() * 100).round(1))
```

실행 결과

```
name
contradiction    29.4
entailment       10.0
neutral           8.8
Name: neg, dtype: float64
```

`|` '또는'. 넷 중 하나라도 있으면 True

빈칸 '안 '은 앞뒤에 빈칸을 둔다. '보안', '동안', '안에'를 빼려고 한다

mean True의 비율

모순 가설의 약 30%에 부정 표현이 있다. 함의와 중립의 세 배다.

기준값은 전제다

노트북 입력 — 작성자가 쓰지 않은 문장에는 얼마나 있나

```
premises = train.drop_duplicates('premise')['premise']
print(len(premises), (premises.str.contains(NEG).mean() * 100).round(1))
```

실행 결과

```
8387 11.7
```

문장	부정 표현이 있는 비율
전제 (찾은 말)	11.7%
가설: 함의	10.0%
가설: 중립	8.8%
가설: 모순	**29.4%**

15강의 ‘무엇에 비해서’다. 함의와 중립은 전제와 비슷한데 모순만 튀어나온다.

글자로 잡은 것을 확인한다

노트북 입력 — 10강의 형태소로 다시 센다

```
from kiwipiepy import Kiwi
kiwi = Kiwi()
NEG_TOKENS = {('않', 'VX'), ('없', 'VA'), ('못', 'MAG'), ('안', 'MAG')}
def has_neg(s):
    return any((t.form, t.tag) in NEG_TOKENS for t in kiwi.tokenize(s))
train['kneg'] = train['hypothesis'].map(has_neg)
print((train.groupby('name')['kneg'].mean() * 100).round(1).to_dict())
```

실행 결과

```
{'contradiction': 27.2, 'entailment': 8.9, 'neutral': 7.8}
```

글자로는 ‘못박혀’, ‘없애달라’까지 잡고 ‘안먹었습니다’를 놓친다. 형태소로 세면 2%p쯤 내려가지만 **모양은 그대로다**.

전제에 부정이 있으면

노트북 입력

```
train['pneg'] = train['premise'].str.contains(NEG)
both = train.groupby(['pneg', 'name'])['neg'].mean().unstack()
print((both * 100).round(1))
```

실행 결과

name	contradiction	entailment	neutral
pneg			
False	29.8	5.5	6.6
True	26.3	43.2	26.0

False 전제에 없는 부정이 모순에서만 새로 들어온다(29.8% 대 5.5%, 6.6%)

True 함의는 전제의 부정을 옮겨 적는다(43.2%). 모순은 지우기도 한다

검증 세트에서도 그럴까

노트북 입력 — 같은 셈을 다른 분할에

```
valid = nli['validation'].to_pandas()
valid['name'] = valid['label'].map(names)
valid['neg'] = valid['hypothesis'].str.contains(NEG)
table = pd.DataFrame({'train': train.groupby('name')['neg'].mean(),
                      'validation': valid.groupby('name')['neg'].mean()})
print((table * 100).round(1))
```

실행 결과

name	train	validation
contradiction	29.4	14.6
entailment	10.0	10.0
neutral	8.8	8.1

함의와 중립은 그대로인데 **모순만 절반으로** 줄었다. 모순 가설이 1,000개이니 우연이라 보기 어렵다.

출처가 달라서일까

노트북 입력 — 모순만 출처별로

```
def by_source(df):
    c = df[df['name'] == 'contradiction']
    return c.groupby('source')['neg'].mean()
src = pd.DataFrame({'train': by_source(train),
                    'valid': by_source(valid)})
print((src * 100).round(1).T)
```

실행 결과

source	NSMC	airbnb	policy	wikinews	wikipedia	wikitree
train	30.8	30.9	22.9	33.1	31.1	26.8
valid	19.5	14.5	10.0	16.0	12.7	13.3

영화평과 숙소평의 몫은 40%로 훈련 세트(39%)와 비슷한데, **출처 여섯 곳 모두** 줄었다.

논문에 답이 있다

Park et al. (2021) 21쪽

"Note that we **deliberately** form the development and test sets in a way to 1) contain balanced source styles and 2) **disincentivize models exploiting annotation artifacts**."

일부러 그렇게 나눈 것이다

검증과 시험 세트는 주석의 흔적에 기대는 모델이 덕을 보지 못하도록 구성했다. 오늘 센 수는 그 설명과 잘 맞는다.

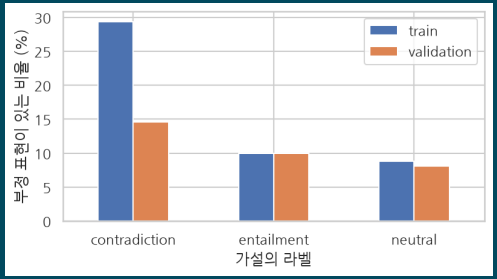
분할은 무작위 표본이라는 보장이 없다

이름이 `train`, `validation`이라고 같은 모집단의 두 조각은 아니다. **분할마다 따로 세어 보고**, 다르면 합치기 전에 까닭을 찾는다.

막대그림으로

노트북 입력

```
ax = (table * 100).plot.bar(figsize=(6, 3), rot=0)
ax.set_xlabel('가설의 라벨')
ax.set_ylabel('부정 표현이 있는 비율 (%)')
plt.savefig('plots/klue-negation.png', dpi=150, bbox_inches='tight')
```



막대 한 쌍의 차이를 읽는다. 왼쪽 쌍만 크게 다르다. 다른 두 쌍은 분할이 달라도 거의 같다. 줄어든 것은 모순의 부정 표현뿐이다.

‘부정이면 모순’이라는 규칙

노트북 입력 — 부정 표현이 있는 가설 가운데 라벨의 몫

```
for df in [train, valid]:
    share = pd.crosstab(df['neg'], df['name'], normalize='index')
    print(share.loc[True].round(3).to_dict())
```

실행 결과

```
{'contradiction': 0.615, 'entailment': 0.211, 'neutral': 0.173}
{'contradiction': 0.446, 'entailment': 0.306, 'neutral': 0.248}
```

`normalize='index'`는 행마다 합이 1이 되도록 나눈다. 모순은 어느 분할에서나 전체의 3분의 1쯤이다. 전제를 읽지 않고 ‘부정이면 모순’이라 찍으면 훈련 세트에서는 61.5%가, 검증 세트에서는 44.6%가 맞는다. 18·19강의 분류기도 이런 규칙을 배울 수 있다.

언어 자료로 쓸 때

말할 수 없는 것

‘한국어 문장의 30%에 부정 표현이 있다’가 아니다. 가설은 과제를 받고 쓴 문장이고, 분할에 따라 수도 달라진다.

말할 수 있는 것

이런 지시를 받은 작성자들이 모순을 어떻게 쓰는가, 곧 부정과 모순에 대한 **작성자의 전략**을 볼 수 있다. 반의어(‘따뜻하게’ → ‘차갑게’)나 수량, 때를 바꾸는 쪽과 비교해 볼 수도 있다.

데이터셋을 쓰기 전에 물을 것

누가 썼나, 무엇을 시켰나, 어떻게 나누었나. 언어학자에게는 잡음이 아니라 자료가 되기도 하지만, 그러려면 만든 과정을 알아야 한다.

확인 질문

여기까지 정리

1. `load_dataset('klue/klue', 'nli', split='validation')`에서 인자 세 개는 각각 무엇을 정하는가?
2. `revision`을 적지 않은 코드가 내년에 다른 수를 낼 수 있는 까닭은?
3. `datasets`를 설치했더니 `str.count(r'\b완전\b')`가 0이 된 까닭과, 고치는 방법은?
4. 가설의 부정 표현을 전제와 비교한 까닭은? 전제 안에서도 조심할 점은?
5. 훈련 세트의 29.4%만 보고 '모순 문장의 특징'을 말하면 무엇이 문제일까?

1 HuggingFace Hub

2 코드로 불러오기

3 KLUE-NLI 읽기

4 **가져다 쓰기 전에**
점검표와 라이선스
직접 해 보기
저장소에 남기기

5 마무리

데이터셋 점검표

	물음	오늘의 KLUE-NLI
1	무엇을, 어디서 모았나	말뭉치 6개의 전제, 사람이 쓴 가설
2	누가, 어떤 지시로 만들었나	작성자 11명, 지침(논문 20–21쪽)
3	어떻게 나누었나	평가 세트는 일부러 구성, 시험 세트는 Hub에 없음
4	불러와지는가	Parquet, `load_dataset` 한 줄
5	어떤 버전인가	커밋 `349481e`
6	써도 되는가	CC BY-SA 4.0
7	직접 읽어 보았나	세 가지 관계의 예, 부정 표현 세기

기말프로젝트에서

이 일곱 가지 물음에 답하지 못하는 데이터셋은 보고서에 쓰기 어렵다. 답을 노트북 첫 칸에 적어 두자.

라이선스

표시	뜻	Hub의 예
CC0	조건 없음	(10강 NSMC의 원본)
BY	출처를 밝힌다	Bingsu/zeroth-korean
SA	고친 것도 같은 조건으로	klue/klue
NC	비영리만	heegyu/namuwiki-extracted
ND	고친 것을 배포하지 않는다	KorQuAD/squad_kor_v1
(없음)	허락받았다고 볼 수 없다	HAERAE-HUB/KOREAN-WEBTEXT

결과를 공개할 때 조건이 따라온다

SA면 내가 만든 표도 같은 라이선스로 내놓고, ND면 고친 데이터를 올리지 않는다. 모두의 말뭉치(11강)는 신청서에 적은 대로 `data/` 아래에만 두었다.

태그는 사람이 적은 것이다

데이터셋	Hub의 태그	원본 또는 카드 본문
e9t/nsmc	cc-by-2.0	원본 GitHub README: CC0 1.0
wicho/kor_3i4k	cc-by-4.0	카드 본문: CC BY-SA 4.0

어느 쪽을 따를까

Hub의 저장소는 원저자가 아닌 사람이 옮겨 올렸을 수도 있고, 태그는 그때 손으로 적는다. 두 곳이 다르면 **원저자가 낸 곳**(논문, 원본 저장소)을 확인한다. 그래도 알 수 없으면 더 엄격한 쪽으로 쓰거나 저자에게 묻는다.

KLUE도 카드의 Licensing 칸은 비어 있고 태그와 논문에 적혀 있었다. 한 곳만 보지 않는다.

출처 적기

기말프로젝트 보고서에 적을 것

Park, Sungjoon et al. (2021). KLUE: Korean Language Understanding Evaluation. arXiv:2105.09680.
HuggingFace Hub `klue/klue`, 설정 `nli`, 분할 `train`, `validation`, 커밋 `349481e`.
CC BY-SA 4.0. 2026-11-09 내려받음.

적을 것	까닭
논문 또는 원저자	저자의 공로. BY의 조건
저장소, 설정, 분할	같은 표를 찾아가는 길
커밋	같은 내용인지 확인하는 길
라이선스	내 결과물에 따라오는 조건

14강의 사전 버전 `20261012`와 같은 일이다. 코드에는 `revision`으로, 보고서에는 글로 남긴다.

실습

wicho/kor_3i4k를 읽어 보자

서울말 화자 세 명이 짧은 발화를 일곱 가지 범주(조각, 진술, 질문, 명령, 수사 의문, 수사 명령, 억양에 달린 발화)로 나눈 데이터셋이다.

1. https://huggingface.co/datasets/wicho/kor_3i4k의 카드를 열고 점검표 1–3, 6번의 답을 찾는다.
2. load_dataset으로 불러와 분할별 행의 수를 찍는다.
3. 분할별로 라벨의 비율을 표로 만든다. KLUE-NLI처럼 분할 사이가 다른가?
4. 라벨 6(intonation-dependent utterance)의 텍스트를 네 개 읽고, 왜 글자만으로는 정할 수 없는지 말해 보자.

오늘 쓴 코드를 고쳐 쓰면 된다. 5분쯤 해 본 다음 답을 함께 본다.

실습 풀이 (1) 불러오고 읽기

노트북 입력

```
k3 = load_dataset('wicho/kor_3i4k')
print({s: d.num_rows for s, d in k3.items()})
t3 = k3['train'].to_pandas()
print(t3.loc[t3['label'] == 6, 'text'].head(4).tolist())
```

실행 결과

```
{'train': 55134, 'test': 6121}
['누구 와요', '누가 절 찾아요', '겨울의 꽃샘추위도 다 가고 잘 지내세요', '잘 있었어']
```

‘누구 와요’는 억양에 따라 설명 의문(who), 판정 의문(someone), 진술로 읽힌다.
글자에는 억양이 없다. 이 카드는 만든 과정까지 채워져 있다(주석자 세 명, 일치도 0.85).

실습 풀이 (2) 분할별 비율

노트북 입력

```
n3 = k3['train'].features['label'].int2str
share3 = {s: k3[s].to_pandas()['label'].value_counts(normalize=True)
          for s in k3}
print((pd.DataFrame(share3).rename(index=n3) * 100).round(1).head(5))
```

실행 결과

	train	test
label		
statement	29.9	29.9
question	29.2	29.2
command	21.2	21.2
fragment	9.8	9.8
intonation-dependent utterance	5.4	5.3

두 가지 분할의 비율이 거의 같다. 비율을 맞추어 나눈 듯하다. 나누는 방법도 데이터셋마다 다르다.

기말프로젝트를 위해 둘러볼 곳

저장소	내용	불러오기
klue/klue	과제 8개, 뉴스 제목, 의존 구문	된다
wicho/kor_3i4k	화행 일곱 가지 범주	된다
SpellOnYou/kor_sarcasm	트윗의 반어 여부	된다
heegyu/korean-petitions	국민청원 436,660건	된다
Moo/korean-parallel-corpora	한영 병렬 문장	된다
moon1ite/kor_hate	혐오 표현 주석	오류

이번 주에 할 일

[숙제10]의 주제에 쓸 데이터셋을 찾아 점검표를 채운다. 오류가 나면 카드의 원본 링크를 따라가고, Hub에 없으면 17강에서 직접 모은다.

무엇을 남길까

터미널 입력 — 노트북이 목록에 없으면 Ctrl+S로 저장부터 한다

```
cd /workspaces/LDS2026
git status
```

실행 결과

```
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        16-huggingface.ipynb
        plots/klue-negation.png

nothing added to commit but untracked files present (use "git add" to track)
```

커밋하기

터미널 입력

```
git add 16-huggingface.ipynb plots/klue-negation.png
git commit -m "16강 실습 노트북: KLUE-NLI 가설의 부정 표현"
```

실행 결과

```
[main a1b2c3d] 16강 실습 노트북: KLUE-NLI 가설의 부정 표현
 2 files changed, 684 insertions(+)
 create mode 100644 16-huggingface.ipynb
 create mode 100644 plots/klue-negation.png
```

내려받은 데이터는 저장소 밖 `~/.cache`에 있어 목록에 없었다. 올리지 않아도 `revision`이 같은 내용을 가리킨다.

푸시하기

터미널 입력

```
git push
```

실행 결과

```
Enumerating objects: 7, done.
Writing objects: 100% (5/5), 39.17 KiB | 39.17 MiB/s, done.
To https://github.com/your-username/LDS2026.git
   e4f5a6b..a1b2c3d  main -> main
```

재현에 필요한 것이 모두 남았다

저장소 이름, 설정, 커밋(revision), 부정 표현의 정의(NEG), 패키지 버전이 노트북에 있다. `string_storage` 한 줄까지 있어야 다른 사람도 같은 수를 얻는다.

1 HuggingFace Hub

2 코드로 불러오기

3 KLUE-NLI 읽기

4 가져다 쓰기 전에

5 **마무리**
- 요약
- 다음 시간
- 과제

오늘 쓴 것

하는 일	오늘의 코드
불러오기	`load_dataset('klue/klue', 'nli')`
버전 정하기	`load_dataset(…, revision='349481e…')`
분할 하나만	`load_dataset(…, split='validation')`
라벨 번호를 이름으로	`d.features['label'].int2str`
표로 바꾸기	`d.to_pandas()`
엔진 되돌리기	`pd.set_option('mode.string_storage', 'python')`
중복 없애기	`s.drop_duplicates()`

표로 바꾼 뒤로는 12강의 `str.contains`, 7강의 `groupby`, 15강의 기준값 그대로다.

오늘 배운 것

Hub와 카드

- 저장소는 올린 이/데이터셋이다
- 카드는 `README.md`이고 빌 수 있다
- 비면 논문과 원본으로 간다

불러오기

- 저장소, 설정, 분할, `revision`
- 스크립트뿐이면 불러오지 못한다
- 설치 하나가 수를 바꿀 수 있다

KLUE-NLI

- 전제는 찾은 말, 가설은 시켜서 쓴 말
- 모순의 부정: 29.4% 대 14.6%
- 평가 세트는 일부러 구성했다

가져다 쓰기

- 일곱 가지 물음의 점검표
- 태그와 원본의 라이선스를 함께 본다
- 커밋까지 출처에 적는다

확인 질문

오늘 것을 정리해 봅시다

1. 카드의 Dataset Creation이 비어 있을 때 어디를 찾아가는가?
2. `e9t/nsmc`가 불러와지지 않는 까닭은? 그 데이터는 어떻게 얻는가?
3. KLUE-NLI의 훈련 세트와 검증 세트를 합쳐 부정 표현을 세면 무엇을 놓치는가?
4. 3i4K와 KLUE-NLI는 분할의 성격이 어떻게 다른가?
5. 여러분의 기말프로젝트 데이터에 대해 점검표의 일곱 가지 물음에 답해 보자. 어느 칸이 비는가?

다음 시간에 배울 것 (11월 11일 수요일)

제17강 코퍼스 구축과 웹 크롤링

원하는 데이터가 어디에도 없다면 직접 모은다. 웹 페이지에서 텍스트를 뽑아 코퍼스를 만들고, 그 과정을 남이 다시 할 수 있게 적는다.

미리 생각해 둘 것

오늘의 점검표를 이번에는 내가 답할 차례다. 내가 만드는 코퍼스의 카드에는 무엇을 적을 수 있을까? 어떤 페이지를 모아도 되고, 어떤 것은 안 될까?

과제 안내

[숙제12] 기말프로젝트 중간발표 자료 [보고서]

마감 **2026-11-13 12:00 (금요일 정오)**. 발표는 11월 16일(월)과 18일(수) 두 번에 나누어 한다. 데이터를 소개하는 쪽에 오늘의 점검표와 출처를 넣자.

[숙제00심화] 논문의 통계량 · 시각화 재검토 (선택)

오늘 분할마다 따로 세어 본 것처럼, 읽은 논문의 수를 데이터로 다시 살펴보자. eTL 심화과제 게시판에 공개글로, 12월 18일 09:00까지 수시로 올린다.